

Redukcja wymiarów $6D \rightarrow 4D$ z wykorzystaniem współrzędnych barycentrycznych i wyznaczników

Roman Kotlowski

June 21, 2026

Abstract

W pracy przedstawiono numeryczną metodę redukcji wymiarów z opisu opartego na parach (AL_i, AN_i) do reprezentacji w przestrzeni czterowymiarowej. Metoda wykorzystuje współrzędne barycentryczne oraz wyznaczniki macierzy 4×4 .

1 Wprowadzenie

W wielu zagadnieniach obliczeniowych pojawia się potrzeba redukcji wymiarów — sprowadzenia złożonego opisu wielowymiarowego do postaci bardziej zwartej, lecz nadal zachowującej kluczowe własności geometryczne i relacje pomiędzy danymi. W prezentowanej metodzie punktem wyjścia jest sześciowymiarowy opis stanu, reprezentowany przez trzy pary parametrów (AL_i, AN_i) , które można interpretować jako współrzędne w dwóch niezależnych osiach dla trzech punktów pomiarowych lub trzech cech systemu.

Celem metody jest przekształcenie tego opisu do reprezentacji czterowymiarowej, w której każdy punkt opisany jest współczynnikami (R_1, R_2, R_3, R_4) . Współczynniki te pełnią rolę współrzędnych barycentrycznych w prostokącie czterowymiarowym i stanowią zwartą, numerycznie stabilną reprezentację danych wejściowych.

Proces redukcji przebiega dwuetapowo. W pierwszym kroku każda para (AL_i, AN_i) jest odwzorowywana na trójkę (X_i, Y_i, Z_i) , spełniającą warunek barycentryczny $X_i + Y_i + Z_i = 1$. Transformacja ta odpowiada rzutowaniu punktu na dwuwymiarowy prostokąt (trójkąt) i stanowi lokalne spłaszczenie danych wejściowych.

W drugim kroku z trzech uzyskanych trójek (X_i, Y_i, Z_i) budowana jest macierz główna M_{main} oraz cztery macierze pomocnicze, w których odpowiednie kolumny zastępowane są wartościami związanymi z układem barycentrycznym. Wyznaczniki tych macierzy — W oraz WX_1, WX_2, WX_3, WX_4 — pozwalają wyznaczyć współczynniki:

$$R_k = \frac{WX_k}{W}, \quad k = 1, 2, 3, 4.$$

Współczynniki te pełnią rolę współrzędnych w przestrzeni czterowymiarowej i stanowią końcowy wynik redukcji. Metoda ta jest deterministyczna, oparta na klasycznych narzędziach algebry liniowej i nie wymaga iteracyjnych procedur optymalizacyjnych.

2 Metoda

2.1 Współrzędne barycentryczne

Każda para (AL, AN) jest przekształcana do trójki (X, Y, Z) spełniającej:

$$X + Y + Z = 1.$$

2.2 Macierze i wyznaczniki

Macierz główna ma postać:

$$M_{\text{main}} = \begin{bmatrix} 1 - X_1 & -X_1 & -X_1 & 0 \\ 0 & 1 - X_2 & -X_2 & -X_2 \\ 1 - X_3 & 0 & -X_3 & -X_3 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

Współczynniki redukcji:

$$R_k = \frac{W X_k}{W}.$$

3 Implementacja w Pythonie

```
import math
import tkinter as tk
from tkinter import ttk

# 1. Function: mapping (AL, AN) -> (X, Y, Z)
def barycentric(AL2, AN2):
    sqrt3 = math.sqrt(3)
    sqrt075 = math.sqrt(0.75)

    DEN = (-1/(2*sqrt075))*1 - (1/(2*sqrt075))*1

    # X
    X = (
        (
            ((sqrt3/2) * AN2 + (1/(2*sqrt075)) * 0.5 * AN2) * 1
            - (((sqrt3)/2) * (1-AL2) - (1/(2*sqrt075)) * (1 - 0.5*(1-
                AL2))) * 1
        ) / DEN * -1
    )
    -
    (
        (-1 / -1.7320508076) *
        (
            (-1/(2*sqrt075)) *
            (((sqrt3)/2) * (1-AL2) - (1/(2*sqrt075)) * (1 - 0.5*(1-AL2)
                )))
            -
            (1/(2*sqrt075)) *
            (((sqrt3/2) * AN2) + (1/(2*sqrt075)) * 0.5 * AN2)
        )
        / DEN
    )
)
```

```

# Y
Y = (
    (
        (-1/(2*sqrt(0.75))) *
        (((sqrt(3)/2) * (1-AL2) - (1/(2*sqrt(0.75))) * (1 - 0.5*(1-AL2)))
        -
        (1/(2*sqrt(0.75))) *
        (((sqrt(3)/2) * AN2) + (1/(2*sqrt(0.75))) * 0.5 * AN2)
    )
    / DEN
) / math.sin(math.radians(60))

Z = 1 - X - Y
return X, Y, Z

# 2. Determinant of 4x4 matrix
def det4(M):
    def det3(A):
        return (
            A[0][0]*(A[1][1]*A[2][2] - A[1][2]*A[2][1])
            - A[0][1]*(A[1][0]*A[2][2] - A[1][2]*A[2][0])
            + A[0][2]*(A[1][0]*A[2][1] - A[1][1]*A[2][0])
        )

    det = 0
    for col in range(4):
        sub = []
        for r in range(1, 4):
            row = []
            for c in range(4):
                if c != col:
                    row.append(M[r][c])
            sub.append(row)

        det += ((-1)**col) * M[0][col] * det3(sub)

    return det

# 3. GUI + calculations
def calculate():
    try:
        AL = [float(al1.get()), float(al2.get()), float(al3.get())]
        AN = [float(an1.get()), float(an2.get()), float(an3.get())]

        X = []
        Y = []
        Z = []

        for i in range(3):
            x, y, z = barycentric(AL[i], AN[i])
            X.append(x)

```

```

        Y.append(y)
        Z.append(z)

# Main matrix (W)
M_main = [
    [1 - X[0], -X[0], -X[0], 0],
    [0, 1 - X[1], -X[1], -X[1]],
    [1 - X[2], 0, -X[2], -X[2]],
    [1, 1, 1, 1]
]
W = det4(M_main)

# WX1
M_WX1 = [
    [0, -X[0], -X[0], 0],
    [0, 1 - X[1], -X[1], -X[1]],
    [0, 0, -X[2], -X[2]],
    [1, 1, 1, 1]
]
WX1 = det4(M_WX1)

# WX2
M_WX2 = [
    [1 - X[0], 0, -X[0], 0],
    [0, 0, -X[1], -X[1]],
    [1 - X[2], 0, -X[2], -X[2]],
    [1, 1, 1, 1]
]
WX2 = det4(M_WX2)

# WX3
M_WX3 = [
    [1 - X[0], -X[0], 0, 0],
    [0, 1 - X[1], 0, -X[1]],
    [1 - X[2], 0, 0, -X[2]],
    [1, 1, 1, 1]
]
WX3 = det4(M_WX3)

# WX4
M_WX4 = [
    [1 - X[0], -X[0], -X[0], 0],
    [0, 1 - X[1], -X[1], 0],
    [1 - X[2], 0, -X[2], 0],
    [1, 1, 1, 1]
]
WX4 = det4(M_WX4)

if W != 0:
    R1 = WX1 / W
    R2 = WX2 / W
    R3 = WX3 / W
    R4 = WX4 / W
else:

```

```

        R1 = R2 = R3 = R4 = float('nan')

    output = ""
    for i in range(3):
        output += f"X{i+1}={X[i]:.10f}, Y{i+1}={Y[i]:.10f}, Z{i+1}={Z[i]:.10f}\n"

    output += "\nWyznaczniki:\n"
    output += f"W (glowny) = {W:.10f}\n"
    output += f"WX1 = {WX1:.10f}\n"
    output += f"WX2 = {WX2:.10f}\n"
    output += f"WX3 = {WX3:.10f}\n"
    output += f"WX4 = {WX4:.10f}\n"

    output += "\nIlorazy WX/W:\n"
    output += f"WX1/W = {R1:.10f}\n"
    output += f"WX2/W = {R2:.10f}\n"
    output += f"WX3/W = {R3:.10f}\n"
    output += f"WX4/W = {R4:.10f}\n"

    result_box.config(state="normal")
    result_box.delete(1.0, tk.END)
    result_box.insert(tk.END, output)
    result_box.config(state="disabled")

except ValueError:
    result_box.config(state="normal")
    result_box.delete(1.0, tk.END)
    result_box.insert(tk.END, "Bład: wpisz poprawne liczby.")
    result_box.config(state="disabled")

# 4. GUI
root = tk.Tk()
root.title("Konwersja 6D -> 4D + Wyznaczniki WX1-WX4 + Ilorazy WX/W")

frame = ttk.Frame(root, padding=20)
frame.grid()

ttk.Label(frame, text="AL1").grid(column=0, row=0)
ttk.Label(frame, text="AN1").grid(column=1, row=0)
ttk.Label(frame, text="AL2").grid(column=0, row=1)
ttk.Label(frame, text="AN2").grid(column=1, row=1)
ttk.Label(frame, text="AL3").grid(column=0, row=2)
ttk.Label(frame, text="AN3").grid(column=1, row=2)

al1 = ttk.Entry(frame); al1.grid(column=0, row=0, padx=5)
an1 = ttk.Entry(frame); an1.grid(column=1, row=0, padx=5)
al2 = ttk.Entry(frame); al2.grid(column=0, row=1, padx=5)
an2 = ttk.Entry(frame); an2.grid(column=1, row=1, padx=5)
al3 = ttk.Entry(frame); al3.grid(column=0, row=2, padx=5)
an3 = ttk.Entry(frame); an3.grid(column=1, row=2, padx=5)

```

```
ttk.Button(frame, text="Oblicz", command=calculate).grid(column=0, row=3,
    columnspan=2, pady=10)

result_box = tk.Text(frame, width=70, height=30, state="disabled")
result_box.grid(column=0, row=4, columnspan=2)

root.mainloop()
```

4 Podsumowanie

Przedstawiona metoda umożliwia deterministyczną redukcję wymiarów z przestrzeni opisanej parami (AL_i, AN_i) do reprezentacji czterowymiarowej (R_1, R_2, R_3, R_4) .